



Microservices Practical / Hands-on Questions

Service-to-Service REST Communication & API Gateway

Question 1 — Service-to-Service REST Communication

Use Case: Online Payment Validation

An e-commerce application has two independent microservices:

- Customer Service — port 8081
- Payment Service — port 8082

When a customer makes a payment, Payment Service must verify that the customer exists and is active before continuing.

Task

Customer Service:

- Create GET /api/customers/C101
- Return customer information:

```
{  
  "customerId": "C101",  
  "name": "Rahul", "status":  
  "ACTIVE"  
}
```

Payment Service:

- Create GET /api/payments/validate/C101
- Use Spring RestClient to call Customer Service.
- If status is ACTIVE, return: Customer C101 is eligible for payment
- Otherwise return: Customer C101 is not eligible for payment

Hints

1. Create two separate Spring Boot applications.
2. Run Customer Service on 8081 and Payment Service on 8082.
3. Create a local CustomerResponse DTO in Payment Service.



4. Create CustomerClient in Payment Service.
5. Inject and use RestClient.
6. Call `http://localhost:8081/api/customers/{customerId}`.
7. Read the returned status and produce the eligibility result.
8. Keep customer business logic in Customer Service and payment logic in Payment Service.



Expected Flow

Client
|
Payment Service :8082
|
| RestClient Customer
Service :8081
|
Customer Response
|
Payment Service
|
Eligibility Result

Question 2 — API Gateway Routing

Use Case: Online Learning Platform

An online learning platform has two independent microservices:

- Student Service — port 8081
- Course Service — port 8082

Introduce an API Gateway so clients use one common entry point instead of knowing individual service ports.

Task

Create an API Gateway on port 8080 with these routes:

Gateway Path	Destination
/api/students/**	Student Service :8081
/api/courses/**	Course Service :8082

Student endpoint:

GET /api/students/S101

Expected:

```
{  
  "studentId": "S101",  
  "name": "Amit",  
  "course": "Java Full Stack"  
}
```

Course endpoint:



GET /api/courses/C101

Expected:

```
{
  "courseId": "C101", "courseName":
  "Java Full Stack", "duration": "6
  Months"
}
```

Hints

9. Create Student Service on 8081 and Course Service on 8082.
10. Create a separate Spring Boot API Gateway on 8080.
11. Use Spring Cloud Gateway Server Web MVC.
12. Configure routes in application.properties.
13. Use the Student route:

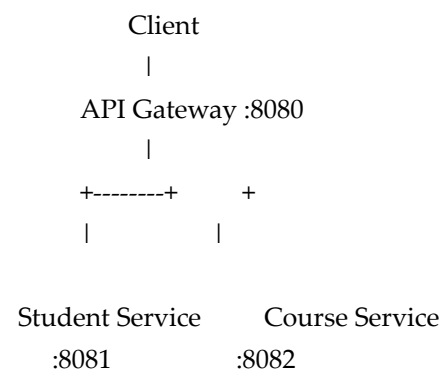
```
spring.cloud.gateway.server.webmvc.routes[0].id=student-service
spring.cloud.gateway.server.webmvc.routes[0].uri=http://localhost:8081
spring.cloud.gateway.server.webmvc.routes[0].predicates[0]=Path=/api/students/**
```

Configure the Course route similarly:

```
spring.cloud.gateway.server.webmvc.routes[1].id=course-service
spring.cloud.gateway.server.webmvc.routes[1].uri=http://localhost:8082
spring.cloud.gateway.server.webmvc.routes[1].predicates[0]=Path=/api/courses/**
```

14. Test through port 8080, not directly through 8081/8082.
15. Keep business logic inside Student/Course services, not the Gateway.

Expected Flow



Final Test

Test: <http://localhost:8080/api/students/S101>

<http://localhost:8080/api/courses/C101>



The client should use port 8080 rather than directly calling ports 8081 or 8082.

Learning Focus

- Service-to-service REST communication using RestClient.
- Clear service boundaries and local DTOs.
- API Gateway as a single entry point.
- Path-based Gateway routing.
- Keeping business logic out of the Gateway.